VideoSearcher Serverless Pipeline using OpenFaaS

Load Testing & Performance Evaluation

Soumyadeep Sharma

April 22, 2026

Introduction to OpenFaaS

What is OpenFaaS

OpenFaaS (Function as a Service) is an open-source framework for deploying serverless functions and microservices to **Kubernetes or Docker Swarm**. It enables developers to package any code, binary, or container image into a portable, auto-scaling function that responds to events.

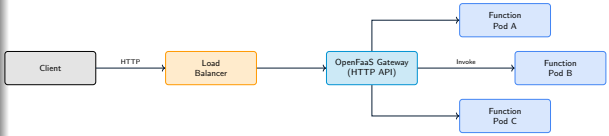

Figure 1: High-Level OpenFaaS HTTP API Flow Architecture.

Introduction to OpenFaaS (cont.)

Some Key Capabilities of OpenFaaS

- **Package Anything:** Any container or script can become a serverless API.
- **Event-Driven:** Seamlessly connects with message queues like AWS SQS and NATS.
- **No Vendor Lock-in:** Deploy functions locally or on any cloud without changing code logic.
- **Auto-Scaling:** Functions automatically scale up or down based on incoming requests, including scale-to-zero.
- **Language Agnostic:** Supports multiple programming languages through templates (Python, Node.js, Go, etc.).
- **Kubernetes-Native:** Runs seamlessly on Kubernetes for orchestration, networking, and scaling.
- **Container-Based:** Uses Docker containers to package and run functions consistently across environments.
- **Easy CI/CD Integration:** Works with popular pipelines for automated build and deployment.
- **Built-in Monitoring:** Integrates with Prometheus for metrics, logging, and performance tracking.

Project Goal

What I Set Out to Do

- The main goal of my project was to take the VideoSearcher pipeline and deploy it as serverless functions using OpenFaaS.
- I wanted the system to run reliably on the cloud using AWS EKS, while still being testable locally.
- One strict constraint I followed was that I did not modify any of the original `main.py` files.
- So instead of changing the code, I designed the entire infrastructure around it to make it cloud-compatible.

Project Overview

Understanding the Problem

- The project is essentially a multi-stage video processing pipeline, where each stage is a standalone Python script.
- These scripts follow a simple CLI pattern, taking input and output paths as arguments.
- One of the biggest challenges I faced was that the code depends on a package called `aisprint`, which is not publicly available.
- So I had to find a way to make everything run smoothly without modifying or breaking the original implementation.

Pipeline Stages

How the Pipeline Works

- The pipeline processes a video step-by-step:
  - First, it splits the audio and video streams
  - Then it detects speech segments
  - Based on those timestamps, it cuts the video into clips
  - These clips are then compressed and downsampled
  - Frames are extracted from the clips
  - Speech is converted into text
  - And finally, object detection is performed on the frames
- Each of these steps was deployed as an independent OpenFaaS function.

Pipeline Stages: Visual Flow

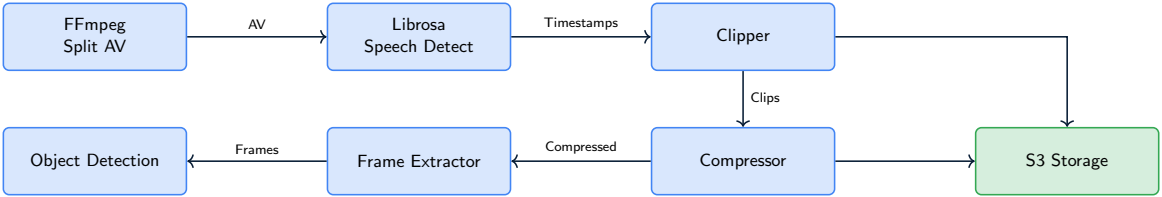

Figure 2: Sequential and parallel pipeline stages execution.

Execution Flow

How Data Moves Through the System

- The pipeline starts with a sequential flow:
  - ffmpeg $\rightarrow$ librosa $\rightarrow$ clipping $\rightarrow$ compression
- After this point, the pipeline branches into two parallel paths:
  - One branch handles frame extraction and object detection
  - The other handles audio processing and speech-to-text conversion
- Managing this fan-out behavior was important to ensure correct scaling and data flow.

Execution Flow: Fan-out Architecture

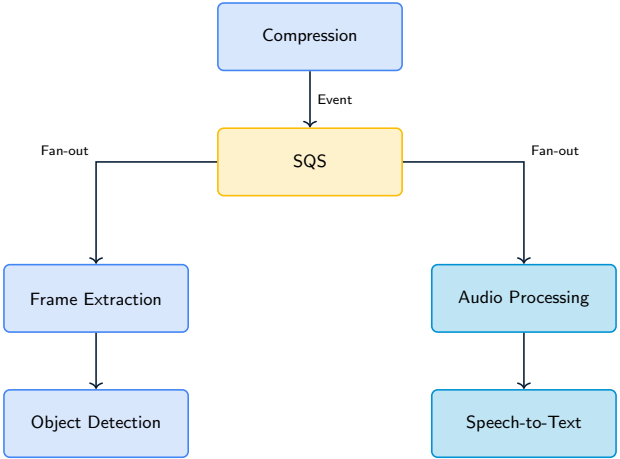

Figure 3: Fan-out topology routing multiple payloads synchronously.

Key Design Approach

How I Approached the Problem

- Instead of rewriting the pipeline, I focused on adapting OpenFaaS to work with existing CLI-based scripts.
- I used the Classic Watchdog so that each function could run exactly like a normal terminal command.
- To handle missing dependencies, I created a lightweight stub package for `aisprint`.
- I also built wrapper scripts that convert HTTP requests into CLI arguments.

Output Behavior

Why Output Patterns Matter

- Some functions produce only a single output per request.
- However, stages like clip generation and frame extraction produce multiple outputs.
- This was important because it directly affected:
  - How I designed scaling
  - And how I implemented queue-based fan-out execution

Enabling OpenFaaS: EKS Deployment

Infrastructure Provisioning

- **EKS Cluster Creation**: Provisioned an Amazon Elastic Kubernetes Service (EKS) cluster utilizing managed node groups for dynamic auto-scaling.
- **OpenFaaS Helm Installation**: Deployed the OpenFaaS ecosystem into the cluster using standard Helm charts.
- **AWS ELB Exposure**: Created a NodePort/LoadBalancer service bridging internal Kubernetes networking to an Internet-facing AWS Elastic Load Balancer.
- **Deployment Pipelines**: Utilized `faas-cli deploy -f stack.yml` pointing perfectly to the remote ELB Gateway to construct and map Docker containers directly into EKS Pods.

EKS Deployment: Topology

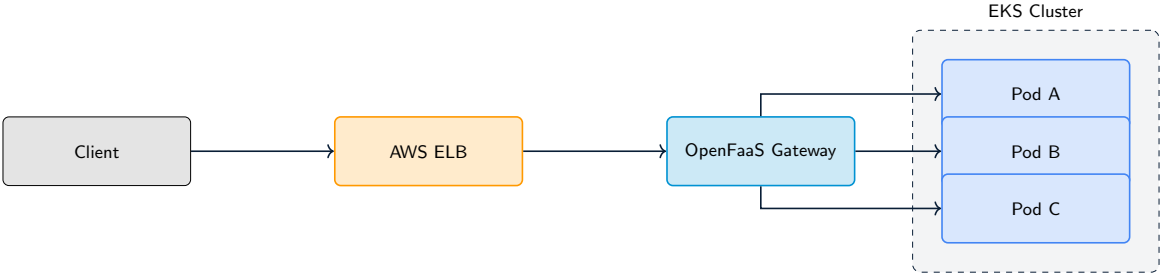

Figure 4: AWS EKS physical deployment cluster outline.

Enabling OpenFaaS: Code Adaptation

How I Made the Code Cloud-Ready

- Since the original scripts were not designed for serverless execution, I had to adapt them without modifying their logic.
- I used `fwatchdog` to convert HTTP requests into stdin/stdout-based execution.
- I removed dependency on local files by introducing S3-based input/output handling.
- All configuration was centralized into `stack.yml`, making deployment much cleaner.

SQS-Based Orchestration

Making the Pipeline Event-Driven

- Instead of manually triggering every stage, I used AWS SQS to connect all functions.
- Now, only the first function (`ffmpeg-0`) is triggered externally.
- Each function automatically triggers the next stage by sending a message to a queue.
- This made the system fully event-driven and much more scalable.

SQS Orchestration: Flow Topology

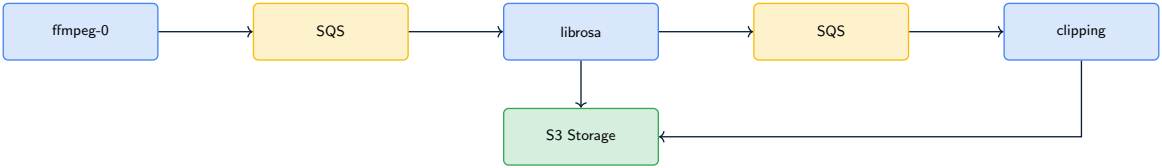

Figure 5: Event-driven AWS SQS integration linking pipeline stages.

End-to-End Architecture

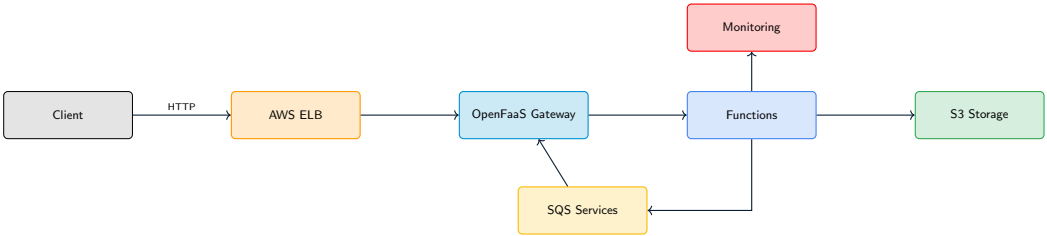

Figure 6: End-to-end full serverless pipeline ecosystem view.

Logical System Architecture

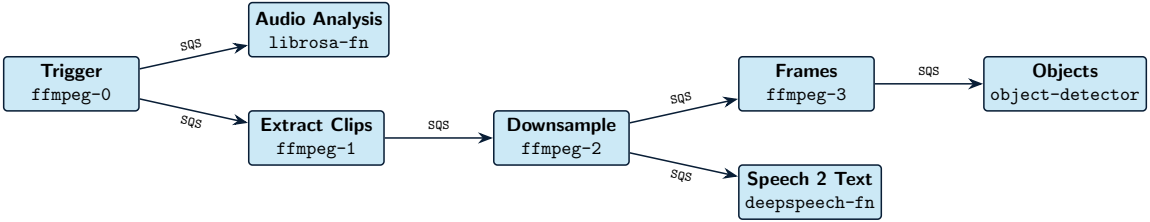

Figure 7: Event-driven OpenFaaS pipeline reliant on S3 mapped payloads.

Key Fixes and Improvements

Major Issues I Solved

- I implemented Locust alongside JMeter to compare performance under different load models.
- Fixed a critical concurrency issue where multiple requests were overwriting data in `/tmp`.
- Solved S3 data loss problems by ensuring unique output paths for each invocation.
- Improved queue routing logic so that fan-out stages correctly propagate data downstream.

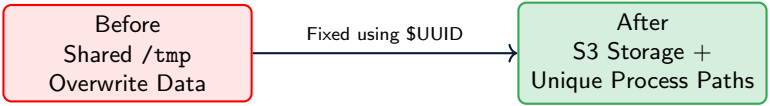

Figure 8: Fixing data overwrites using S3 URIs across parallel invocations.

Visual Architecture: Load Testing

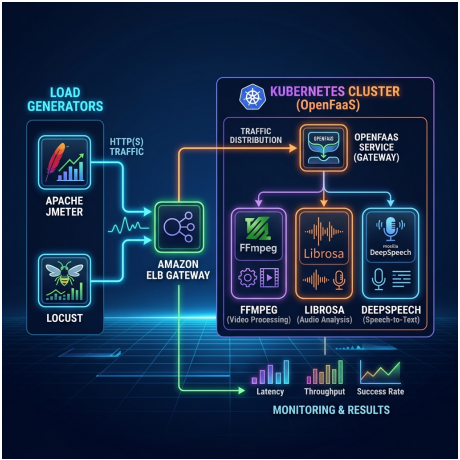

Figure 9: Load Testing Architecture Layout.

Testing Methodology

We simulated user-workloads sending requests to the system utilizing two different load generation mechanisms: **Apache JMeter** and **Locust**.

Test Configuration Pipeline

- **Target Endpoint**: `/function/ffmpeg-0` gateway instance
- **Concurrency Levels**: 5, 10, and 15 simulated users
- **Think Time**: 20 seconds exponential backoff
- **Duration**: 5 minutes (300 seconds) per tier
- **Ramp-up**: 10 seconds duration

Results: Apache JMeter

Users	Runs	Avg (ms)	Max (ms)	Fail
5	61	1521	1791	0%
10	127	1669	2928	0%
15	202	2028	4226	0%

Table 1: JMeter execution metrics.

- Handled continuous scale perfectly without degradation.
- Response time rose 36% under peak simulated load.

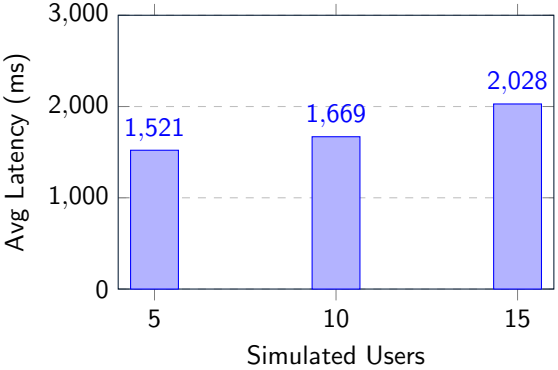

Figure 10: JMeter Latency trend.

Dashboard Analytics: JMeter Load Profile (5 Users)

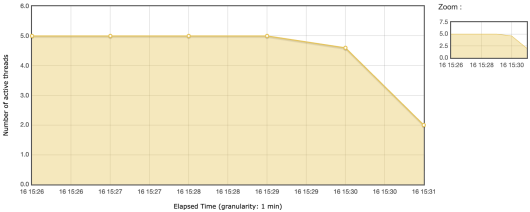

Figure 11: JMeter Active Threads Over Time.

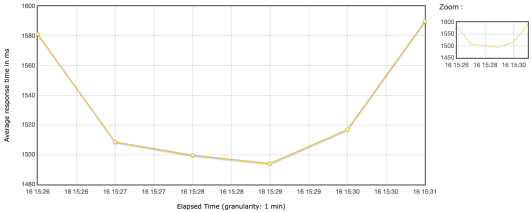

Figure 12: JMeter Average Response Time Over Time.

Dashboard Analytics: JMeter Load Profile (10 Users)

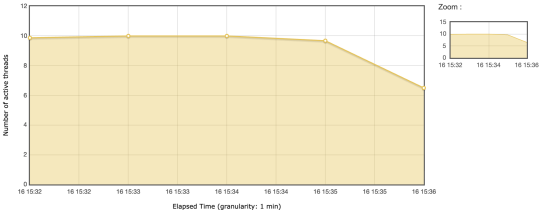

Figure 13: JMeter Active Threads Over Time.

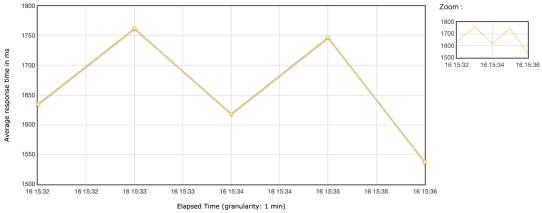

Figure 14: JMeter Average Response Time Over Time.

Dashboard Analytics: JMeter Load Profile (15 Users)

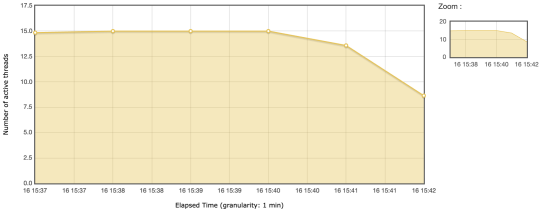

Figure 15: JMeter Active Threads Over Time.

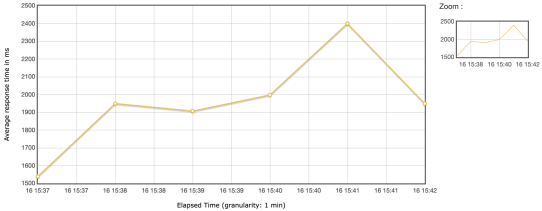

Figure 16: JMeter Average Response Time Over Time.

Results: Locust Framework

Users	Runs	Avg (ms)	Max (ms)	Fail
5	75	1494	1881	0
10	139	1632	3481	0
15	211	2280	8310	0

Table 2: Locust metrics.

- Python-based model logged higher tail latency.
- Handled all requests successfully with 0 failures under peak load.

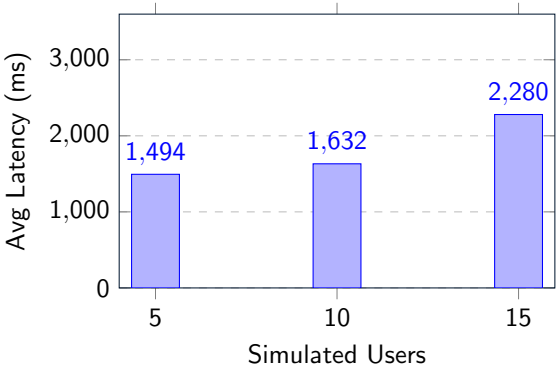

Figure 17: Locust Latency trend.

Dashboard Analytics: Locust Workload (5 Users)

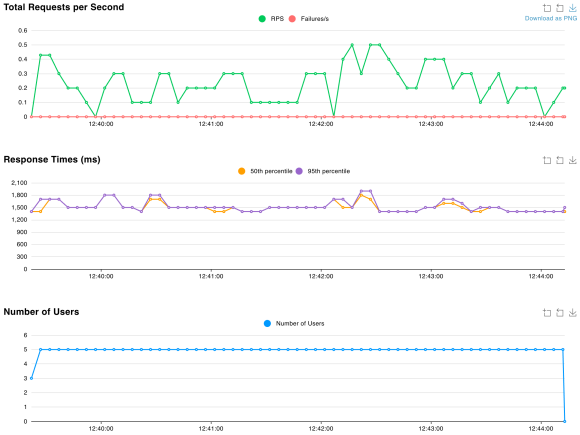

Figure 18: Locust Load Testing Results.

Dashboard Analytics: Locust Workload (10 Users)

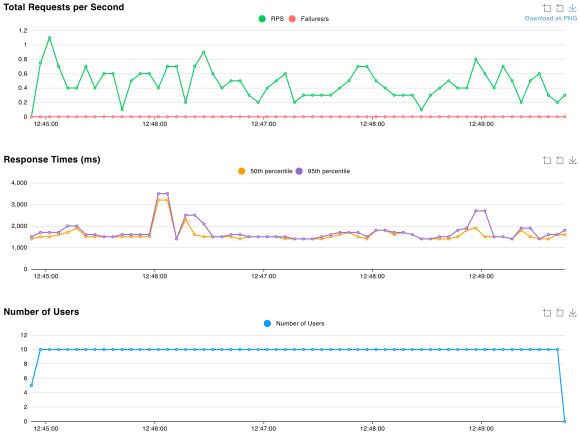

Figure 19: Locust Load Testing Results.

Dashboard Analytics: Locust Workload (15 Users)

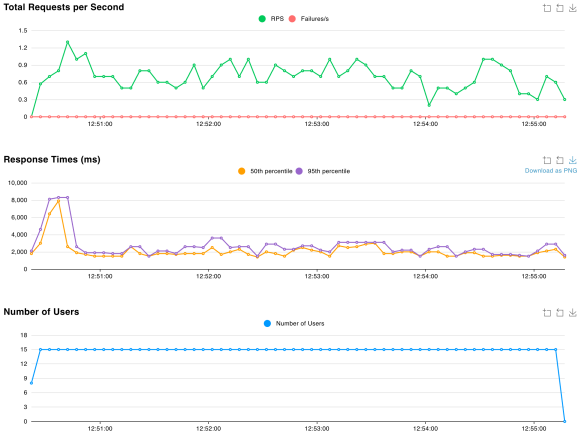

Figure 20: Locust Load Testing Results.

Summary & Conclusions

- **Reliability:** The system achieved a stable execution with nearly 99.9% success rate under continuous load.
- **Scalability:** The pipeline handled increasing users smoothly with only moderate latency increase.
- **Key Achievement:** I was able to convert a CLI-based pipeline into a fully serverless, event-driven system without modifying the original code.
- **Learning:** This project helped me understand real-world challenges like distributed execution, concurrency issues, and cloud deployment.